Republic of the Philippines
POLYTECHNIC UNIVERSITY OF THE PHILIPPINES
College of Computer and Information Sciences
Sta.Mesa, Manila

CASE STUDY DOCUMENTATION

Relic Hunter: The Escaping Thief

A* Search & Minimax Search with Alpha-Beta Pruning

In partial fulfillment for the Course:

COSC 304 – Introduction to Artificial Intelligence

Names of the Proponents:

Cabaddu, Elija C.

Jocson, Dan Louie M.

Lambohon, Ashley H.

Salgado, Samantha Marion C.

A.Y. 2025-2026

TABLE OF CONTENTS

1. Introduction and its Background

The effectiveness of a game is not solely determined by its visual design or mechanics, but more importantly by the intelligence and responsiveness of the system that interacts with the player. In many traditional games, non-player characters (NPCs) rely on predefined patterns or predictable behaviors, which often leads to repetitive gameplay and reduced player engagement over time. According to Zhang (2024), the integration of artificial intelligence in game environments enables the development of more adaptive, strategic, and responsive systems that enhance player experience and immersion.

One of the most relevant applications of artificial intelligence in game development is in pursuit-based scenarios, where an AI agent is tasked with locating and intercepting a moving target within a structured environment such as a maze. These scenarios require efficient navigation and adaptability to dynamic player actions. As noted by Shevtekar et al. (2022), artificial intelligence techniques are widely used in adversarial environments to simulate intelligent and competitive behavior between agents.

In relation to this, pathfinding and decision-making are essential components in creating intelligent game agents. These allow AI-controlled entities to determine effective movement within a grid-based environment while considering obstacles and environmental constraints. According to Zhang (2024), AI-based search techniques are commonly applied in games to support navigation efficiency and adaptive behavior in complex environments.

This study presents **Relic Hunter: The Escaping Thief**, a turn-based, grid-based stealth or chase game that applies artificial intelligence concepts to create an engaging and strategic gameplay experience. In the game, the player assumes the role of a thief who must navigate a maze-like environment while carrying a stolen relic and attempt to reach a designated exit tile. Although the objective appears simple, the presence of an AI-controlled guard transforms the

experience into a strategic challenge that requires careful planning and decision-making.

The core premise draws inspiration from classic chase games (most notably the ghost-pursuit mechanic of Pac-Man), but to increase the complexity and unpredictability of the game, the player is given the ability to place temporary obstacles within the environment. These obstacles dynamically alter available paths within the maze, directly affecting both player strategy and AI behavior.

Unlike fixed-pattern enemies, the AI guard uses a hybrid system where **Minimax** with **Alpha-Beta Pruning** handles strategic decision-making, while **A* Search** handles real-time pathfinding and movement execution. This dual-layered system ensures that the AI is not only efficient in finding the shortest path but also strategic in countering the player's moves. The setting is a temple or dungeon, but the focus is squarely on gameplay mechanics: escape, evasion, and resource management.

Through the integration of these techniques, the study aims to demonstrate how combining pathfinding and decision-making algorithms can produce a more intelligent and adaptive game system. Ultimately, this research showcases the practical application of AI in creating strategic depth and unpredictability in interactive, dynamic environments.

2. Game Proper

2.1. Objective of the Game

The main objective of Relic Hunter is for the player to successfully navigate a maze-like environment and reach the designated exit point while carrying the stolen relic. The player wins only when they occupy the "exit" tile in the grid like a maze. However, the game ends in failure when the AI guard, powered by the A* Search and Minimax Search with Alpha-Beta Pruning algorithm, catches the player by being in the same

grid tile. In order for the player to win, they must balance their movement and blocking strategies with the placement of obstacles to delay the AI from reaching the player.

2.2. Rules of the Game

The game operates on a turn-based, grid-based system where every move from the player counts. The core mechanics of the game are as follows:

2.2.1. Three-Round Progression

The game consists of three rounds. The player or AI wins the game by successfully winning 2 out of 3 or all 3 rounds. To ensure variability and prevent predictable strategies, the maze layout is procedurally altered each round. This includes changes in:

- Wall Placement
- Path Configuration
- Exit Tile Location

Furthermore, each round introduces progressive difficulty scaling. While the core AI implementation and game mechanics remain unchanged, several gameplay parameters are adjusted to increase challenge. These include:

Parameter	1st Round (Easy)	2nd Round (Medium)	3rd Round (Difficult)
Guard speed (tiles per turn)	1	1.25	1.5
Barricade duration (turns)	5	4	3
Maximum active barricades	4	3	2
Minimax search depth	1	2	3

Map layout complexity	Wide corridors, fewer dead ends; 9×9 grid	Mixed corridors, moderate branching; 12×12 grid	Narrow corridors, complex branching; 15×15 grid

2.2.2. Grid and Movement

The game world is a 2D tile-based grid. On each turn, the player may move one tile in one of the four cardinal directions (up, down, left, right), provided the destination tile is not a wall or occupied barricade.

2.2.3. Player Actions

On the player's turn, they are allowed to perform one of two actions:

- **Move:** Move the current location of the player to an adjacent, accessible tile that is not blocked by obstacles.
- **Distraction:** Place a temporary obstacle on an adjacent, accessible tile (including diagonals) to block the AI's path. This consumes the player's entire turn. Additionally, the obstacles can only be placed on open tiles and cannot overlap with existing walls or the AI itself. It also disappears after a fixed number of turns, after which it becomes passable again.

2.2.4. AI Behavior (Hybrid Search)

After the player's turn (move or barricade placement), the AI guard takes its turn. The AI guard cannot see the exit tile, and its only objective is the player's current coordinates.

- **Prediction:** Using the Minimax with Alpha-Beta Pruning, the AI guard thinks of the player's potential movements to determine the best direction or path to follow the player.

- **Pathfinding:** Once the AI guard decides on its next move, A* Search makes sure that it takes the most efficient path around permanent walls and player-placed obstacles.

2.2.5. Movement Limitations

Both the player and the AI are restricted by the maze's boundaries. Neither can pass through permanent walls or active obstacles.

2.2.6. Core Gameplay Loop

- **Round Win:** The player moves onto the designated exit tile.
- **Round Loss:** The AI guard moves onto the tile currently occupied by the player.
- **Match Win:** The player or the AI wins 2 out of 3 rounds or all 3 rounds.

3. Implementation of an AI Algorithm

3.1. Chosen algorithms and Rationale

The guard's intelligence is governed by three complementary techniques: A* Search, Minimax with α–β Pruning. The table below summarises each algorithm's role:

Algorithm	Purpose	Advantage	Role in Guard AI
A* Search	Finds the shortest path on the grid from the guard to the player each turn	Optimal and complete; uses an admissible Manhattan distance heuristic; efficient re-computation per turn	Pathfinding component (executed every guard turn)
Minimax	Adversarial look-ahead simulating guard and player moves	Anticipates player blocking; not purely greedy	Strategic decision-selection

α–β Pruning	Prunes Minimax branches that cannot affect the final outcome	Dramatically reduces nodes evaluated without changing the result; enables deeper look-ahead in real time	Applied within Minimax to optimize performance

3.1.1. A* Search

A* is a highly efficient informed search algorithm critical for real-time video game navigation (Lawande et al., 2022). It evaluates nodes by combining **g(n):** the exact cost to reach the node from the start, and **h(n):** the heuristic estimated cost from the node to the goal (Liu, 2023). In this grid-based game, the heuristic used is the **Manhattan Distance.**

$$f(n) = g(n) + h(n)$$

- **Purpose**: The primary purpose of A* Search is to handle the navigation efficiency of the guard by finding the shortest and most optimal path to the player's current coordinates. It calculates the shortest path from the guard's current position to a target tile (often the player's current coordinate) while avoiding permanent walls and temporary barricades.
- **Rationale:** It is chosen for its speed and accuracy in grid-based environments. The Manhattan distance heuristic (defined as |x_guard − x_player| + |y_guard − y_player|) is used because the game's grid restricts movement to the four cardinal directions, making it both appropriate and admissible (it never overestimates the true cost), which guarantees that A* will always find the optimal path.

When the player places a barricade, the guard simply re-runs A* on the updated grid, instantly computing the new best

path. This re-computation is efficient enough to complete within a single game turn.

3.1.2. Minimax with α–β Pruning

Minimax is a recursive adversarial search technique for two-player, zero-sum games. Alpha-Beta (α-β) pruning is used to remove branches in the search tree that are statistically guaranteed not to impact the final decision. Shevtekar et al. (2022) highlight the usefulness of these algorithms in imitating intelligent and competitive behavior among agents in hostile contexts. Additionally, recent implementations demonstrate that integrating α-β pruning significantly balances computational efficiency with strategic decision-making (Suancha et al., 2024).

In this game, the system models the guard as the maximising player (minimising distance to the player) and the thief as the minimising player (maximising distance from the guard). A depth-limited search (depth 1–3, scaling with round difficulty) allows the guard to anticipate the player's likely blocking moves before committing to a step. The algorithm assumes optimal player behavior, although in practice, the thief is controlled by a human player who may not always make optimal decisions.

- **Purpose:** This algorithm provides tactical anticipation by simulating a game tree of potential future moves from both the guard and the player, including the player's option to place barricades as an alternative to moving.
- **Rationale:** Unlike a purely greedy "chase" AI, Minimax allows the guard to anticipate the player's attempts to block paths with barricades. α–β Pruning is integrated to dramatically reduce the number of nodes evaluated in the

search tree, allowing for deeper look-ahead without impacting real-time performance.

- ***Complements A* Search:*** These algorithms work in a dual-layered pipeline: Minimax with Alpha-Beta Pruning is used to determine the guard's next strategic target position based on simulated player movement, while A* Search is used to compute the optimal path from the guard's current position to that selected target.

3.2. Pseudocode

3.2.1. A* Search

```
function A_STAR(grid, start, goal):
    openSet  ← PriorityQueue()
    openSet.insert(start, 0)
    cameFrom ← {}
    gScore   ← default ∞; gScore[start] ← 0

    while openSet not empty:
        current ← openSet.pop_min()
        if current == goal: return reconstruct_path(cameFrom, current)

        for neighbor in get_neighbors(grid, current):
            g ← gScore[current] + 1
            if g < gScore[neighbor]:
                cameFrom[neighbor] ← current
                gScore[neighbor]   ← g
                if neighbor not in openSet:
                    openSet.insert(neighbor, g + heuristic(neighbor, goal))
    return FAILURE
```

```
function heuristic(a, b): return |a.x-b.x| + |a.y-b.y|

function get_neighbors(grid, node):
    return [node+d for d in [UP,DOWN,LEFT,RIGHT]
            if in_bounds(node+d) AND grid[node+d] is walkable]

function reconstruct_path(cameFrom, current):
    path ← [current]
    while current in cameFrom: current ← cameFrom[current];
path.prepend(current)
    return path
```

A* Search Pseudocode Explanation:

- **Smart Navigation (openSet & cameFrom):** The algorithm prioritizes the most promising paths using a Priority Queue and leaves a breadcrumb trail to reconstruct the final route.
- **Step Counting (gScore):** Tracks the exact cost from the start to each node. The estimated total cost f(n) = g + h is computed on the fly when inserting into the priority queue, rather than stored separately.
- **The Distance Guess (heuristic):** It calculates "Manhattan distance" (grid-based counting) to perfectly estimate the remaining distance, as diagonal movement is restricted.
- **Safe Movement (get_neighbors):** It restricts the guard to legal moves, avoiding walls, edges, and barricades.

3.2.2. Minimax with α–β Pruning

```
function GET_BEST_GUARD_MOVE(gameState, maxDepth):
    bestMove, bestScore ← null, -∞
    for move in get_guard_moves(gameState):
        score ← MINIMAX(apply_move(gameState, move, GUARD), maxDepth-1, false,
                        -∞, +∞)
        if score > bestScore: bestScore, bestMove ← score, move
    return bestMove

function MINIMAX(gameState, depth, isMax, α, β):
    if guard_pos == thief_pos:  return +∞
    if thief_pos == exit_tile:  return -∞
    if depth == 0:              return evaluate(gameState)

    moves   ← get_guard_moves(gameState)  if isMax  else
get_thief_moves(gameState)
    player  ← GUARD                       if isMax  else  THIEF
    best    ← -∞                          if isMax  else  +∞
    update  ← max                         if isMax  else  min

    for move in moves:
        score ← MINIMAX(apply_move(gameState, move, player), depth-1, not
                        isMax, α, β)
        best  ← update(best, score)
        if isMax: α ← max(α, score)
        else:     β ← min(β, score)
        if α >= β: break
    return best

function evaluate(gameState):
```

```
    return -(|guard.x-thief.x| + |guard.y-thief.y|)

function get_thief_moves(gameState):
    return walkable_neighbors(thief_pos) ∪ eligible_barricade_tiles(thief_pos)

function get_guard_moves(gameState):
    return walkable_neighbors(guard_pos)
```

Minimax Pseudocode Explanation:

- **Starting the Simulation (GET_BEST_GUARD_MOVE):** Tests every legal move the guard can make right now, simulates the future outcomes of those moves, and picks the absolute best choice.
- **Playing Both Sides (MINIMAX):** Alternates turns in the simulation. The Guard tries to maximize the score (get closer), while the Thief tries to minimize it (run away or block).
- **Saving Brainpower (α-β Pruning)**: α is the Guard's worst-case scenario, and β is the Thief's. If the simulation hits a path that is clearly worse than a known alternative (α ≥ β), it stops checking that path to save processing power.
- **Scoring the Future (evaluate):** If the simulation looks ahead as far as it can without anyone winning or losing, it scores the board. The closer the Guard is to the Thief, the better the score.
- **Asymmetrical Rules:** The simulation knows the different character rules: the Guard can only walk, but the Thief can either walk or place a temporary barricade.

3.3. Software Architecture

The diagram below shows all major system components and their data flows, grouped into three areas: Player Side, Core Game Loop, and AI Subsystem. The Guard AI subsystem is highlighted at the right, receiving game state from the game loop and returning guard actions each tick.

- **Player Side -** Part of the system that handles everything directly controlled by the player: input, player movements, and interactions.
- **Core Game Loop -** Part of the system that keeps the game running. It repeatedly updates input, game state, locations, and rendering so the game stays synchronized.
- **Guard/AI Side (AI Subsystem) -** Part of the system that is responsible for enemy (guard) behavior. This game proposal includes perception, pursuit decision-making, pathfinding, and action output.

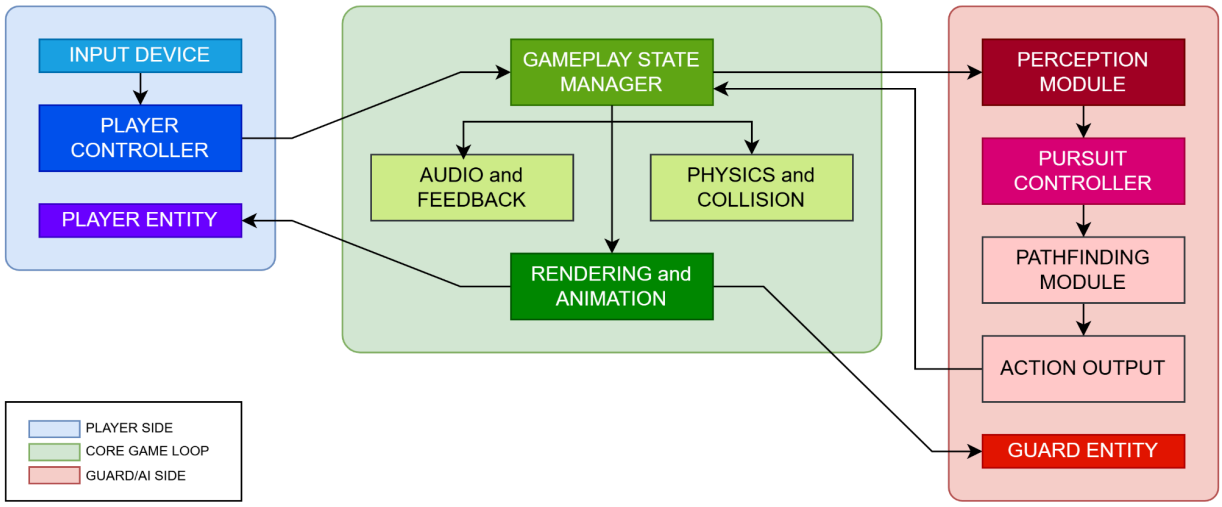

Figure 1. High-Level System Architecture. Boxes are software components, and arrows show data/control flow.

3.4. AI Subsystem Modules

The guard AI is organized as a pipeline comprising four dedicated modules. The full pipeline is illustrated below:

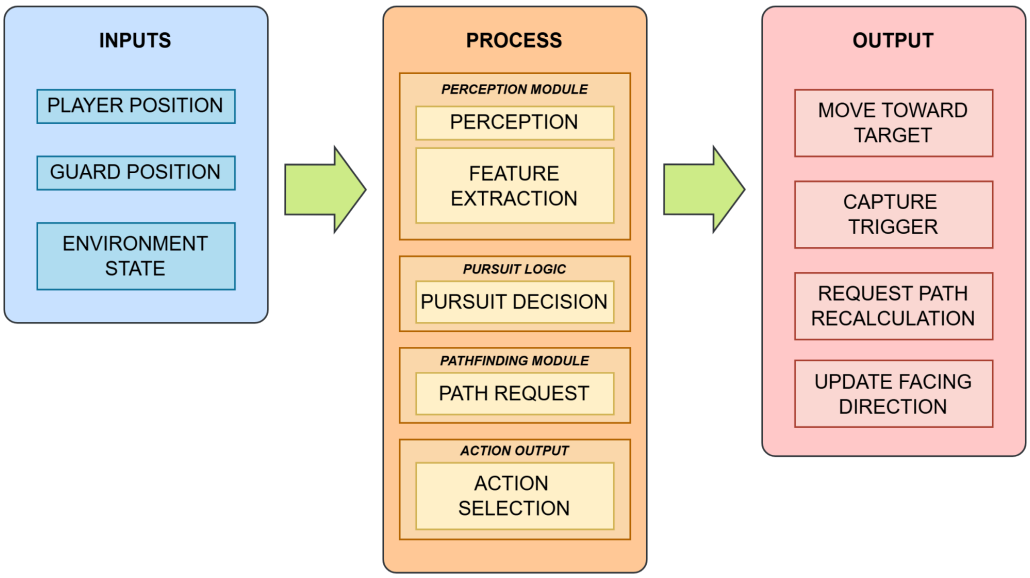

Figure 2. Detailed AI Subsystem Pipeline. The pipeline runs left-to-right in an IPO (Input-Process-Output) model.

3.4.1. Inputs

The set of real-time data received from the game engine that specifies the current state of the environment and the AI's relevant entities. This contains the player's location, guard's location, the distance between entities, and any environmental limitations like barriers or map layout.

3.4.2. Perception and Feature Extraction

This is the process of collecting and translating raw game state data into useful variables for decision-making. Perception reads the current state, which includes player actions, entity

statuses, and environmental state, whereas feature extraction turns structured perception data into usable metrics like distance to player, direction vector, and line-of-sight status.

3.4.3. Pursuit Decision

A lightweight decision-making phase that uses extracted characteristics to predict the guard's immediate behavior. It assesses conditions (for example, closeness to the player) to determine whether to continue chasing or trigger capture.

3.4.4. Path Request

This is the process of creating or modifying a travel path to the destination (player). Based on the pursuit decision, this module queries the pathfinding engine (A* Search and Minimax with α–β pruning algorithm) to provide an ideal route that takes into consideration obstacles and level geometry.

3.4.5. Action Selection

This stage converts the desired behavior and calculated path into executable commands. This involves choosing the right movement direction or interaction action, such as a capture trigger, to send to the game engine.

3.4.6. Outputs

Finally, this is the collection of commands generated by the AI subsystem and sent to the gameplay state manager for execution. These include movement orders, capture events, and orientation changes, which are subsequently handled by the core game loop.

References

Lawande, S. R., Jasmine, G., Anbarasi, J., & Izhar, L. I. (2022). A systematic review and analysis of intelligence-based pathfinding algorithms in the field of video games. *Applied Sciences*, *12*(11), 5499. https://doi.org/10.3390/app12115499

Liu, D. (2023). *Research of the path finding algorithm A* in video games*. *39*, 763–768. https://doi.org/10.54097/hset.v39i.6642

Shevtekar, S., Malpe, M., & Bhaila, M. (2022). Analysis of game tree search algorithms using minimax algorithm and alpha-beta pruning. *International Journal of Scientific Research in Computer Science, Engineering and Information Technology*, 328–333. https://doi.org/10.32628/cseit1228644

Suancha, C. C., Suarez, M. J., & Besoain, F. A. (2024). Implementation of alpha-beta pruning and transposition tables on checkers game. *IEEE Access*, 1–1. https://doi.org/10.1109/access.2024.3381958

Zhang, S. (2024). A study on search techniques in the game-tree. *Applied and Computational Engineering*, *32*(1), 253–258. https://doi.org/10.54254/2755-2721/32/20230221

Appendix A

Gameplay Loop Flowchart

The figure below shows the basic gameplay loop flowchart of the system. The core loop is an interplay of movement and resource management. The player must choose each turn: advance toward the exit, or sacrifice a turn to place a barricade. Each cycle alternates between the player's decision and the guard's AI response, continuing until a win or lose condition is reached.

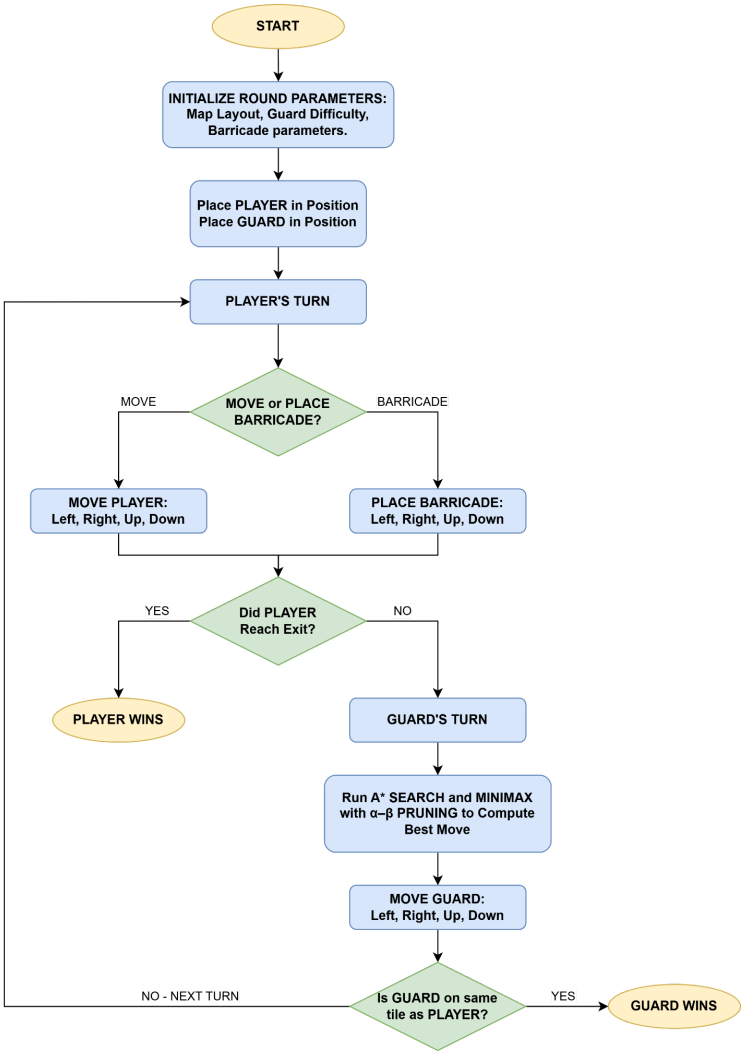

Figure 2. Core Gameplay Loop Flowchart.

Appendix B

Game Concept Mockup

The figure below shows a representative 9×9 grid layout for the first round. It illustrates possible placement of key entities: the thief (player), guard (AI), exit tile, and placed temporary barricades. Actual finalized levels will feature more complex wall configurations, multiple corridors, and carefully designed choke points to support strategic barricade play.

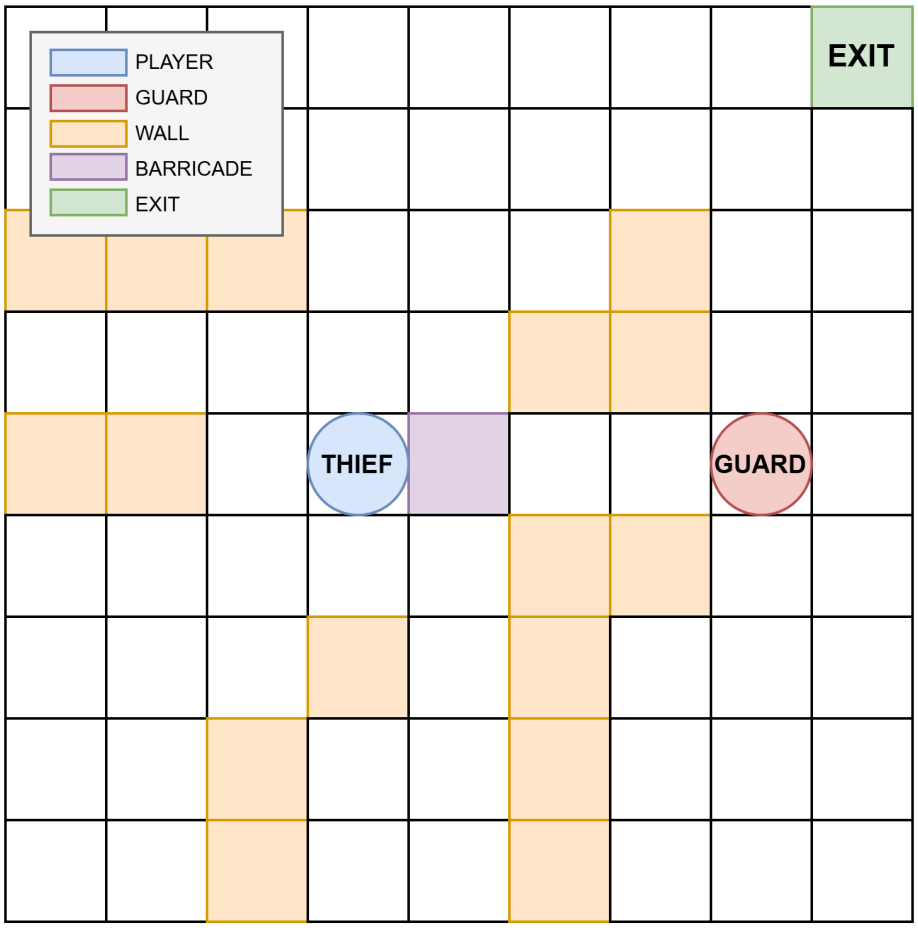

Figure 3. Sample Level Layout (9x9 grid).

Blue = Thief (carries the relic). Red = Guard (AI). Green = Exit tile. Orange tiles = Permanent walls. Purple tile = Active barricade (temporary).

Appendix C

AI Behavior Flow

The behavior of the guard (AI) may be characterized using the finite state machine shown below. The AI guard has three states: idle/patrol, chase/pursue, and attack, with the idle state shifting nearly instantly into pursuit once the theft is detected at the start of the game. In the Chase/Pursue mode, the guard actively chases the player by running the A* and Minimax with Alpha-Beta Pruning algorithms each turn, dynamically recalculating routes when barriers are built. Finally, the Attack state is a terminal condition where the guard reaches the player's tile, resulting in a win for the AI.

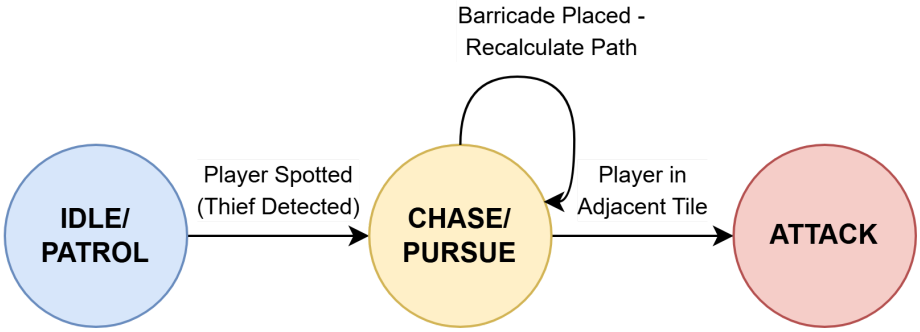

Figure 4. The Finite State Machine for the Guard AI.

Appendix D

Development Timeline

The plan is divided into five sequential phases: Design, Prototyping, Core Build & AI Coding, Testing, and Polishing. The ideation process began on April 13, and the proposal is scheduled to be submitted on April 27, after which the full development begins on April 28. The key milestones and the timeline table are outlined down below.

- **April 13 -** Ideation begins (Week 1 start)
- **April 27 -** Proposal submitted (end of Week 2)
- **April 28 -** Development process begins (Week 3 start)
- **June 1 -** Final project checking (end of Week 7)

Weekly Development Timeline

Week	Dates	Phase	Milestones & Deliverables	Responsible	Stage
1	Apr 13–19	Ideation	• Game concept and core loop definition • Select AI algorithms (A*, Minimax, Minimax with α–β Pruning). • *Deliverable: Game Concept and Chosen Algorithms.*	All Members	Pre-production
2	Apr 20–27	Design Finalization	• Write and finalize Case Study Documentation (Proposal). • Define algorithm interfaces and data structures. • *Deliverable: Submitted Game Dev Proposal.*	All Members	Pre-production
3	Apr 28 – May 4	Prototyping	• Concept Design for Sprites and Levels. • Setup Project Environment and folder structure. • Implement grid data model and tile system.	Jocson - Lead Programmer Lambohon - Game Designer & Art Lead	Build

			• Render basic grid on screen. • *Deliverable: Playable empty grid prototype*	Salgado - Engine Developer	
4	May 5–11	Core Build & AI Coding 1	• Player movement (4-directional) with wall collision. • Barricade placement and TTL expiry system. • Placeholder guard mover. • Turn alternation loop (Player turn → Guard turn). • Implement A* pathfinding algorithm into guard logic. • *Deliverable: Functional turn loop with basic guard movement.*	Jocson - Lead Programmer & AI Programmer Cabbadu - AI Programmer Salgado - Engine Developer	Build
5	May 12–18	AI Coding 2	• Implement Minimax with α–β pruning in guard logic. • Tune search depth for real-time performance. • *Deliverable: Guard navigates maze optimally via A* + Minimax*	Jocson - Lead Programmer & AI Programmer Cabbadu - AI Programmer	AI Implement
6	May 19–25	Testing	• Execute all systematic test cases. • Tune any last-minute changes. • Difficulty balancing across Easy / Medium / Hard. • *Deliverable: Game QA Tested.*	Salgado - QA Tester Lambohon - Game Designer	QA
7	**May 26 – Jun 1**	**Polish & Submission**	• Integrate HUD, sound effects, visual polish. • Finalize art assets and animations. • Add multiple level layouts and difficulty tiers. • Prepare deployment and submission. • *Deliverable: Final Submission Build (June 1)*	All Members Jocson - Lead Programmer Lambohon - Game Designer & Art Lead Salgado - QA Tester	Release

Member Role Distribution		
Role	**Responsibilities**	**Member/s**
Lead Programmer	● Oversees the overall technical direction of the project. ● Integrates the work of the other programmers and ensures all systems connect properly. ● Maintains code consistency, handles major technical decisions, and helps keep the project on schedule.	Dan Louie Jocson
AI Programmer/s	● Implements the guard's AI behavior, including A* Search, Minimax, and the α–β pruning. ● Tunes the AI so it reacts properly to barricades, maze changes, and player movement.	Dan Louie Jocson & Elija Cabaddu
Game Designer	● Plans the level structure, difficulty scaling, and player experience. ● Keeps the development implementation aligned with the game vision.	Ashley Lambohon
Art Lead	● Establishes the visual style of the game. ● Creates or supervises sprites, icons, UI elements, and animations.	
Engine Developer	● Builds the core systems that make the game run. ● Helps manage technical features like game flow, build setup, and integration with the engine.	Samantha Salgado
QA Tester	● Tests the game to check if the mechanics and AI work as intended. ● Records bugs, issues, and unusual behavior during gameplay.	

Appendix E

Tools and Technologies

Component	Tools and Technologies
Algorithms	A* Search, Minimax with α–β Pruning
Game Engine	Unity
Programming Language	C#
Development Environment (IDE)	Visual Studio Code
Art and Animation Tools	Open-source downloadable assets (e.g. OpenGameArt, Kenney.nl, itch.io)
Audio and Sound Design	Open-source downloadable assets (e.g. Freesound.org, OpenGameArt)
Version Control & Project Management	Github
Issue Tracking & QA	GitHub